AI-POWERED INTERVIEW COACH

Project Documentation

An intelligent mock interview platform combining Computer Vision, Natural Language Processing, Speech Recognition, and Generative AI to simulate a realistic interview environment with real-time feedback.

VIDHEESH KUMAR KASANAGOTTU

Infosys AI-Powered Interview Coach Batch- 11(TEAM 1)

GitHub: https://github.com/VIDHEESH/AI_POWERED_INTERVIEW_COACH

Table of Contents

1. Project Overview

2. Approach & Design Philosophy

3. User Flow

4. Features Implemented

5. System Architecture

6. Project Structure & Component Record

7. Technology Stack

8. API Endpoints Reference

9. Database Schema

10. Setup & Installation Instructions

11. Environment Variables

12. Running the Application

13. Privacy & Security

14. Future Enhancements

1. Project Overview

AI-Powered Interview Coach is an intelligent mock interview platform designed to help candidates prepare for technical interviews using real-time AI-driven feedback. The system leverages multiple AI/ML technologies — Computer Vision for eye-contact and emotion analysis, Natural Language Processing for answer evaluation, Speech Recognition for transcription, and Generative AI (Google Gemini) for personalized question generation — all orchestrated across a dual-server architecture running on the user's local machine.

The platform provides a seamless end-to-end experience: users register, upload their resume, paste a job description, receive a TF-IDF based match score, then enter an AI-driven interview session with per-question video recording, real-time gaze monitoring, emotion detection, speech-to-text transcription, and NLP-based scoring of their answers.

2. Approach & Design Philosophy

2.1 Architecture Approach

The project follows a microservice-based architecture with two independent servers working together:

- **Node.js Server (Port 3000):** Handles user authentication, session management, database operations, serves the frontend, manages file uploads, and communicates with Google's Gemini API for question generation and NLP scoring.
- **Python Flask Server (Port 5000):** Handles all heavy AI/ML processing including video analysis (gaze detection via OpenCV Haar Cascades, emotion recognition via FER), audio transcription (OpenAI Whisper), resume–JD matching (TF-IDF + Cosine Similarity), and audio quality analysis (Librosa).

2.2 Design Principles

Principle	Implementation
Privacy-First	Video, audio, and transcription are processed locally on the user's machine. Nothing is sent to the cloud except text prompts for question generation.
Modular	Each AI capability (Gaze Detection, Emotion Analysis, Transcription, JD Matching, Audio Analysis, Video Analysis) is a separate Python module that can be tested, improved, or replaced independently.
Real-Time Feedback	Eye-contact warnings, emotion badges, and live transcription update during the interview, not just as a post-interview summary.
Seamless Flow	The user flow is linear and intuitive: Register → Login → Dashboard → Analyze JD/Resume → Start Interview → Review Results.
Fault Tolerance	Graceful fallbacks at every stage — if Gemini API is unavailable, default questions are used; if emotion detection fails, neutral is assumed; retry logic for API rate limits.

3. User Flow

The application guides users through a structured interview preparation workflow:

Step 1 — Register / Login: User creates an account with name, email, and password (bcrypt hashed). JWT token is issued on login for session management.

Step 2 — Dashboard: Displays all past interview sessions grouped by interview ID, with session history cards showing question text, transcript, NLP scores, and video playback.

Step 3 — Resume & JD Analysis: User pastes the Job Description and uploads their resume (PDF or text). The system calculates a TF-IDF cosine similarity match score and identifies missing keywords.

Step 4 — Proceed to Interview: Using the JD and resume context, the Gemini API generates 5 tailored technical questions. The system falls back to default questions if the API is unavailable.

Step 5 — Real-Time Interview Session: For each question, the camera records video. OpenCV tracks eye contact and the FER library detects emotions in real time. Browser Speech API captures spoken answers. A "LOOK AT CAMERA" warning appears when gaze drifts.

Step 6 — Post-Interview Processing: After all questions are answered, Whisper transcribes each recording locally. Audio analysis measures speaking pace, filler words, pitch, and pauses. Gemini scores each answer on Relevance, Structure, Clarity, and Correctness (1–10 each).

Step 7 — Results Dashboard: Users review per-question transcripts, NLP scores visualized with Chart.js, video playback with analysis overlay, emotion timeline, and actionable tips.

Flow Diagram

```
[Register/Login] → [Dashboard] → [Paste JD + Upload Resume]
        ↓
[TF-IDF Match Score + Missing Keywords]
        ↓
[Proceed to Interview] → [AI generates 5 tailored questions]
        ↓
[Real-time Interview Session]
  • Camera records video per question
  • OpenCV tracks eye contact + emotion
  • Browser Speech API captures answers
  • Whisper transcribes audio locally
        ↓
[Results Dashboard]
  • Per-question transcript
  • NLP scoring (Relevance, Structure, Clarity, Correctness)
  • Video playback with analysis overlay
```

4. Features Implemented

4.1 User Authentication & Authorization

Secure registration and login system using industry-standard practices.

- Registration with name, email, and password validation (minimum 6 characters, valid email regex).
- Passwords hashed with bcrypt (10 salt rounds) before storage.
- JWT tokens issued on login with 24-hour expiry.
- Protected routes using authentication middleware that verifies Bearer tokens.
- Profile view and update functionality.
- Show/hide password toggle on login and registration forms.

4.2 Resume vs Job Description Matching

NLP-based document similarity analysis running entirely locally.

- Uses TF-IDF (Term Frequency–Inverse Document Frequency) Vectorization from scikit-learn.
- Calculates Cosine Similarity between resume and JD to produce a percentage match score.
- Identifies missing keywords from the JD that are absent in the resume.
- Supports PDF upload (parsed via pypdf) and plain text input.
- Text cleaning and normalization (lowercasing, special character removal).
- All processing runs on-device — no data leaves the user's machine.

4.3 AI-Powered Question Generation

Context-aware interview question generation using Google Gemini.

- Sends JD and resume text as context to Google Gemini API (gemini-flash-latest model).
- Generates 5 tailored technical questions specific to the role and candidate background.
- Robust JSON parsing with regex extraction and markdown cleanup.
- Exponential backoff retry logic for API rate limits (429 errors) — 3 retries, 5s/15s/45s delays.
- Fallback to generic default questions if the API is unavailable.
- Supports both specific JD-based and general engineering question modes.

4.4 Real-Time Eye Contact Monitoring

Computer vision-based gaze detection during live interview.

- Uses OpenCV Haar Cascades for face and eye detection (haarcascade_frontalface_default, haarcascade_eye).
- Calculates face center position normalized to frame dimensions (0.0–1.0).
- Detects "Moving Aside" when face center is outside the 15%–85% horizontal range.
- Eye position tracking with standard deviation analysis to detect reading/darting patterns.
- Rolling history buffer (deque, 30 frames) for smoothed gaze state estimation.

- Displays a "LOOK AT CAMERA" warning overlay when gaze deviates.
- Configurable thresholds to minimize false positives.

4.5 Emotion & Confidence Detection

Facial expression recognition for interview confidence assessment.

- Uses the FER (Facial Expression Recognition) library built on TensorFlow/Keras.
- Detects emotions: Happy, Sad, Angry, Surprise, Fear, Disgust, Neutral.
- Displays a floating emotion badge on the video feed during the interview.
- Emotion data is tracked per question for post-interview analysis.
- Graceful fallback — if FER fails to load, emotion defaults to "Neutral" without crashing.
- Post-interview video analysis via DeepFace for dominant emotion across sampled frames.

4.6 Speech-to-Text Transcription

Local audio transcription using OpenAI Whisper.

- Uses OpenAI Whisper (base model) running entirely on the user's machine.
- Forced CPU execution to avoid CUDA/BLAS conflicts between TensorFlow and PyTorch.
- Transcribes each recorded answer after the question recording stops.
- Supports both direct file upload and file path reference modes.
- Full transcript is saved to the PostgreSQL database per question.
- Deferred transcription mode: recordings are saved immediately, transcription runs asynchronously.

4.7 NLP Answer Analysis & Scoring

AI-driven evaluation of candidate answers on multiple dimensions.

- Each transcribed answer is sent to Gemini AI with the question and JD context.
- Scored on four dimensions (1–10 each): Relevance, Structure, Clarity, Correctness.
- Includes a "Key Feedback" one-sentence summary for each answer.
- Prompt engineered to act as a "Senior Technical Interviewer" for realistic assessment.
- Output is strictly validated JSON with markdown cleanup.
- Scores visualized on the dashboard using Chart.js bar charts.

4.8 Per-Question Video Recording

Individual recordings for each interview question.

- Records video and audio per question (not a single long recording).
- Uses MediaRecorder API with WebM format (video/webm;codecs=vp8,opus).
- Each recording saved as a .webm file with unique timestamp-based filename.
- Files stored in the local uploads/ directory and linked to database sessions.

- Video playback available on the dashboard with analysis overlay.
- Interview sessions grouped by a unique interview_group_id (UUID).

4.9 Dashboard & Session History

Comprehensive review of past interview performance.

- Displays all past interview sessions grouped by interview_group_id.
- Each group shows: question text, full transcript, and embedded video playback.
- NLP scores (Relevance, Structure, Clarity, Correctness) visualized with Chart.js.
- Overall metrics: average score, eye contact percentage, dominant emotion, speaking pace.
- Expression/emotion timeline across questions.
- Batch re-processing: re-analyze all past recordings in one click.

4.10 Audio Quality Analysis

Speech quality metrics using Librosa audio processing.

- Speaking Pace: Words per minute (WPM) calculated from transcript word count and audio duration.
- Filler Word Detection: Counts occurrences of "um", "uh", "like", "you know", "actually", "basically".
- Pitch Analysis: Mean fundamental frequency (F0) and standard deviation for monotone detection.
- Pause Detection: Silence ratio using librosa.effects.split with configurable top_db threshold.
- Clarity Score: 0–100 based on deviation from optimal 140 WPM speaking pace.
- Fluency Score: 0–100 with penalties for filler words and excessive pauses.
- Personalized Tips: Context-aware suggestions (slow down, reduce fillers, add vocal variation).

4.11 Video Analysis (Post-Recording)

Frame-by-frame video analysis for behavioral assessment.

- Eye Contact Scoring: MediaPipe Face Mesh for iris and nose tip landmark tracking.
- Dominant Emotion: DeepFace analysis sampled every 30th frame for performance.
- Nervousness Detection: Head pose movement stability scoring (Low/Medium/High).
- Falls back to DeepFace-only mode if MediaPipe is version-incompatible.
- Results merged with audio analysis data for comprehensive per-question feedback.

5. System Architecture

The application is split into three distinct layers that communicate over HTTP:

5.1 Architecture Diagram

```
┌──────────────────────────────────────────────────────┐
│                       BROWSER                        │
│  index.html │ register.html │ dashboard.html         │
│  jd-compare.html │ interview.html │ eye-tracking      │
│           public/js/app.js (Frontend Logic)          │
│           public/css/style.css (Styling)             │
└──────────────────────────────────────────────────────┘
                      │
                      │ HTTP Requests (fetch API)
                      ▼
┌──────────────────────────────┐   ┌──────────────────────────────┐
│    NODE.JS SERVER (:3000)    │   │  PYTHON FLASK SERVER (:5000) │
│                              │   │                              │
│  /auth/* → Authentication    │   │  /compare-jd → TF-IDF Match  │
│  /interview/chat → Gemini AI │   │  /gaze-detect → OpenCV       │
│  /interview/upload → Save    │   │  /transcribe → Whisper       │
│  /interview/sessions → Read  │   │  /analyze-audio → Librosa    │
│  /interview/compare-jd → ────┼──→│  /extract-text → PDF Parse   │
│  /interview/process-batch    │   │  /health → Health Check      │
│  /python/* → Proxy ──────────┼──→│                              │
│                              │   │                              │
│  Database: PostgreSQL        │   │  Models: Whisper, FER, Haar, │
│  Auth: JWT + bcrypt          │   │  DeepFace, MediaPipe         │
└──────────────────────────────┘   └──────────────────────────────┘
```

5.2 Communication Flow

- **Frontend → Node.js:** All HTTP requests use the Fetch API with JWT Bearer tokens in headers.
- **Node.js → Python:** The Node server proxies specific requests (JD compare, transcription) to Flask via internal HTTP calls. A generic /python/* proxy route also forwards any unmatched requests.
- **Node.js → Gemini API:** Text prompts for question generation and NLP scoring are sent to Google Gemini via the @google/generative-ai SDK.
- **Frontend → Python (direct):** Real-time gaze detection frames are sent directly from the browser to the Python service via fetch() for minimal latency.

6. Project Structure & Component Record

Below is a complete record of every file and component in the project, organized by directory and annotated with purpose and key functions.

6.1 Root-Level Files

File	Role	Description
index.js	Main Entry Point	Node.js Express server. Registers middleware (JSON parsing, static files), mounts auth and interview routes, sets up /python/* reverse proxy to Flask, creates uploads/ directory, initializes PostgreSQL and starts listening on PORT.
db.js	Database Connection	Creates PostgreSQL connection pool using pg. Defines initDb() which creates users, questions, and sessions tables. Seeds 5 default interview questions if the questions table is empty.
setup_db.js	DB Bootstrap	Standalone script that connects to PostgreSQL default database and creates the interview_coach database if it does not exist.
package.json	Node Dependencies	Defines project name (interview_coach v1.0.0), entry point (index.js), and 8 dependencies: @google/generative-ai, bcrypt, dotenv, express, jsonwebtoken, multer, pdf-parse, pg.
cleanup_users.js	Utility Script	Helper script for cleaning up user data during development.
.env	Environment Config	Stores PORT, DATABASE_URL, JWT_SECRET, and API_KEY. Not committed to Git.
.gitignore	Git Ignore Rules	Excludes node_modules/, uploads/, .env, __pycache__/, Python model files (.pth), and other generated artifacts.
README.md	Documentation	GitHub README with project overview, features, architecture, setup instructions, and environment configuration.

6.2 Authentication Module (authentication/)

MVC-structured module handling user registration, login, and profile management.

File	Role	Description
route/auth.js	Route Definitions	Defines 4 Express routes: • POST /register — Validates input, hashes password, creates user. • POST /login — Verifies credentials, issues JWT (24h expiry). • GET /profile — Protected; returns user profile. • PUT /profile — Protected; updates user name.
model/user.js	Database Model	Exports 4 functions: • createUser(name, email, passwordHash) • findUserByEmail(email) • findUserById(id) • updateUserProfile(id, name)
middleWares/auth.js	JWT Middleware	Extracts Bearer token from Authorization header, verifies it using JWT_SECRET, attaches decoded user object to req.user. Returns 401 if no token, 400 if invalid.
validation/validate.js	Input Validation	Exports 2 middleware functions: • validateRegister — Checks name, email, password presence; validates email regex; enforces min 6 char password. • validateLogin — Checks email and password presence. Helper: validEmail() uses regex /^[^\s@]+@[^\s@]+\.[^\s@]+$/.

6.3 Interview Module (interview/)

Handles interview sessions, question management, AI chat, and file uploads.

File	Role	Description
route/interview.js	Route Definitions	7 routes + 1 helper:

	(367 lines)	• GET /questions — Returns 5 random questions from DB. • POST /upload — Upload video (.webm), create session, trigger async transcription. • POST /process-batch — Batch transcribe + NLP analyze all sessions in a group. • GET /sessions — Returns all sessions for the authenticated user. • POST /compare-jd — Proxy CV + JD to Python for TF-IDF analysis. • POST /chat — Gemini AI for question generation (isInit) or feedback. • analyzeAnswerWithAI() — Helper: sends Q+A+JD to Gemini for NLP scoring.
model/interview.js	Database Model	Exports 4 functions: • getQuestions() — SELECT random 5 from questions. • createSession(userId, questionId, videoUrl, questionText, interviewGroupId, videoAnalysis) • getUserSessions(userId) — JOIN sessions + questions, ORDER BY created_at DESC. • updateSessionTranscript(id, transcript, analysis)

6.4 Python AI Microservice (python_services/)

Flask-based server for all AI/ML processing tasks.

File	Role	Description
app.py	Flask Entry Point (161 lines)	Loads Whisper model on startup. Defines 5 routes: • /health — Health check. • /transcribe — Whisper transcription + audio/video analysis. • /gaze-detect — Real-time gaze detection from base64 image. • /compare-jd — TF-IDF resume matching. • /extract-text — PDF text extraction. Forces CPU mode (CUDA_VISIBLE_DEVICES=-1).
model_gaze.py	Gaze & Emotion Module (162 lines)	Loads Haar Cascades (face + eye) and FER detector. Exports detect_gaze(image_bgr) which returns: • status: CENTER or AWAY • emotion: Detected expression (Happy, Neutral, etc.) • gaze_state: Focused, Moving Aside, Reading Screen, etc. • eye_coords: Normalized eye positions for UI tracking. Uses deque-based rolling history (30 frames) for smooth detection.
model_jd.py	JD Matching Module (57 lines)	Exports 3 functions: • clean_text(text) — Lowercase + remove special chars. • extract_text_from_pdf_bytes(pdf_bytes) — PDF parsing via pypdf. • compare_jd_cv(jd_text, cv_file_bytes, filename) — TF-IDF + Cosine Similarity, returns (score, missing_keywords, cv_text).
video_analysis.py	Video Analysis Module (146 lines)	Exports analyze_video(video_path): • Eye Contact %: MediaPipe Face Mesh iris + nose landmarks. • Dominant Emotion: DeepFace sampled every 30th frame. • Nervousness Rating: Head movement stability (Low/Medium/High). Falls back to DeepFace-only if MediaPipe is unavailable.
audio_analysis.py	Audio Analysis Module (101 lines)	Exports analyze_audio(file_path, text): • Duration, WPM, Pitch (mean + std), Pause ratio. • Filler word count (um, uh, like, you know, etc.). • Clarity score (0–100) and Fluency score (0–100). • generate_tips() for personalized feedback.
requirements.txt	Python Dependencies	Lists 12 packages: flask, flask-cors, opencv-python-headless, mediapipe, scikit-learn, pypdf, numpy<2, fer, tensorflow, librosa, openai-whisper, deepface, tf-keras.
gaze_model_best.pth	Trained Model Weights	PyTorch model file (~46 MB) for the custom gaze estimation model trained on the Columbia Gaze Dataset.

6.5 Frontend (public/)

Static HTML/CSS/JS frontend served by Express.js.

File	Role	Description
index.html	Login Page	User login form with email/password fields, alert section, show/hide password toggle, and link to registration page.
register.html	Registration Page	Registration form with name, email, password, and confirm password. Input validation and password visibility toggle.
dashboard.html	User Dashboard	Displays interview session history grouped by interview ID. Cards with question text, transcripts, video playback, NLP score charts (Chart.js), and overall performance metrics.
jd-compare.html	JD/Resume Analysis	Job Description text area, resume file upload (PDF/text), TF-IDF match score display (circular score widget), missing keywords list with badges, and "Proceed to Interview" button.
interview.html	Interview Interface	Main interview page (17,600 bytes). Camera preview, question display, recording controls (start/stop/retry/submit), live transcription area, emotion badge overlay, gaze warning overlay, progress indicators.
eye-tracking.html	Eye Tracking Test	Standalone page for testing/previewing eye tracking functionality independently without starting a full interview.
css/style.css	Global Stylesheet (435 lines)	Design system with CSS custom properties (design tokens), Inter font from Google Fonts, glassmorphism card effects, gradient buttons, animated badges, responsive breakpoints, grid background pattern.
js/app.js	Core Frontend Logic (1,377 lines)	Monolithic JavaScript file containing: • Auth: register(), login(), logout(), checkAuth(), token management. • Dashboard: loadSessions(), showInterviewDetail(), calculateMetrics(), renderSpeechChart(), renderExpressionTimeline(). • Interview Engine: sendToAI(), proceedToNextQuestion(), initInterview(), startGazeMonitoring(), attachListeners(), bindInterviewEvents(). • Media: speak() (TTS), camera stream, MediaRecorder, SpeechRecognition. • 58 functions total.

6.6 Other Directories

Directory	Role	Description
uploads/	File Storage	Stores per-question video recordings (.webm files). Created automatically on server startup if missing. Served as static files via /uploads/*.
node_modules/	Node Dependencies	Auto-generated by npm install. Contains all Node.js package dependencies. Excluded from Git.

7. Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript (ES6+)	User interface, page structure, interactivity
Charts	Chart.js	NLP score visualization (bar charts)
Styling	CSS Custom Properties, Google Fonts (Inter)	Design tokens, glassmorphism, responsive layout
Backend	Node.js, Express.js 5.x	REST API server, routing, middleware
Database	PostgreSQL	Persistent storage for users, sessions, transcripts, scores
DB Driver	pg (node-postgres)	PostgreSQL connection pooling and query execution
Authentication	JSON Web Tokens (JWT), bcrypt	Stateless auth, password hashing
File Upload	Multer	Multipart form-data handling, video file storage
PDF Parsing	pdf-parse (Node), pypdf (Python)	Resume text extraction from PDF files
AI Chat	Google Gemini API (@google/generative-ai)	Question generation, NLP answer scoring
AI Server	Python 3.10+, Flask, Flask-CORS	ML model hosting, REST endpoints
Computer Vision	OpenCV (Haar Cascades)	Real-time face and eye detection for gaze monitoring
Face Mesh	MediaPipe Face Mesh	Iris landmark tracking for eye contact scoring in video
Emotion (Real-time)	FER (TensorFlow/Keras)	Facial expression recognition during live interview
Emotion (Post)	DeepFace	Dominant emotion analysis from recorded video frames
Transcription	OpenAI Whisper (base model)	Local speech-to-text for interview answers
Audio Analysis	Librosa	Pitch, pace, pause, and filler word analysis
Text Analysis	scikit-learn (TF-IDF + Cosine Similarity)	Resume vs Job Description matching
Speech (Browser)	Web Speech API (SpeechRecognition)	Live in-browser speech capture
TTS (Browser)	Web Speech API (SpeechSynthesis)	Question read-aloud via browser TTS
Environment	dotenv	Environment variable management

8. API Endpoints Reference

8.1 Authentication Endpoints (Node.js — /auth)

Method	Endpoint	Auth Required	Description
POST	/auth/register	No	Register a new user with name, email, password.
POST	/auth/login	No	Authenticate user and receive JWT token.
GET	/auth/profile	Yes	Get authenticated user's profile.
PUT	/auth/profile	Yes	Update user's name.

8.2 Interview Endpoints (Node.js — /interview)

Method	Endpoint	Auth	Description
GET	/interview/questions	Yes	Get 5 random seeded questions from database.
POST	/interview/upload	Yes	Upload per-question video recording. Creates session in DB. Triggers async Whisper transcription.
POST	/interview/process-batch	Yes	Batch transcribe + NLP analyze all sessions in a group. Used at end of interview.
GET	/interview/sessions	Yes	Get all interview sessions for the logged-in user.
POST	/interview/compare-jd	Yes	Upload CV + JD text. Proxied to Python for TF-IDF scoring.
POST	/interview/chat	Yes	Send text to Gemini AI. If isInit=true, generates 5 questions. Otherwise gives feedback on answer.

8.3 Python AI Service Endpoints (Flask — Port 5000)

Method	Endpoint	Auth	Description
GET	/health	No	Returns {"status": "ok"} — health check.
POST	/transcribe	No	Accepts file upload or filePath. Runs Whisper transcription + audio + video analysis. Returns transcript text and analysis object.
POST	/gaze-detect	No	Accepts base64 image in JSON body. Returns gaze status (CENTER/AWAY), emotion, gaze_state, and eye coordinates.
POST	/compare-jd	No	Accepts CV file + jobDescription text. Returns TF-IDF match score, missing keywords, and extracted CV text.
POST	/extract-text	No	Accepts file upload (PDF or text). Returns extracted plain text.

9. Database Schema

PostgreSQL database: interview_coach

9.1 Users Table

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-incrementing user ID
name	VARCHAR(100)		User's display name
email	VARCHAR(100)	UNIQUE, NOT NULL	User's email address (unique login identifier)
password	VARCHAR(255)	NOT NULL	bcrypt-hashed password
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation time

9.2 Questions Table

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-incrementing question ID
category	VARCHAR(50)		Category: Behavioral or Technical
text	TEXT	NOT NULL	The interview question text

9.3 Sessions Table

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-incrementing session ID
user_id	INTEGER	REFERENCES users(id)	Foreign key to users table
question_id	INTEGER	REFERENCES questions(id)	FK to questions (null for AI-generated)
video_url	TEXT	NOT NULL	Path to stored .webm recording (/uploads/...)
question_text	TEXT		AI-generated question text (when question_id is null)
interview_group_id	TEXT		UUID grouping all questions in one interview session
transcript	TEXT		Whisper-generated transcript of the answer
analysis_results	JSONB		Combined analysis: NLP scores + audio metrics + video metrics
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Session creation time

10. Setup & Installation Instructions

10.1 Prerequisites

- **Node.js:** Version 16 or higher (with npm).
- **Python:** Version 3.10 or higher (with pip).
- **PostgreSQL:** Running database server (any recent version).
- **Google Gemini API Key:** Required only for AI question generation and NLP scoring.
- **Webcam/Microphone:** Required for the interview recording features.

10.2 Installation Steps

Step 1: Clone the Repository

```
git clone https://github.com/VIDHEESH/AI_POWERED_INTERVIEW_COACH.git
cd AI_POWERED_INTERVIEW_COACH
```

Step 2: Install Node.js Dependencies

```
npm install
```

Step 3: Install Python Dependencies

```
cd python_services
pip install -r requirements.txt
cd ..
```

Step 4: Create PostgreSQL Database

```
CREATE DATABASE interview_coach;
```

Step 5: Configure Environment Variables

```
Create a .env file in the project root (see Section 11).
```

Step 6: Initialize Database Tables

```
node setup_db.js
```

11. Environment Variables

Create a .env file in the project root with the following variables:

```
PORT=3000
DATABASE_URL=postgresql://postgres:your_password@localhost:5432/interview_coach
JWT_SECRET=your_secret_key
API_KEY=your_google_gemini_api_key
```

Variable	Required	Default	Description
PORT	Yes	3000	Port for the Node.js server.
DATABASE_URL	Yes	—	Full PostgreSQL connection string.
JWT_SECRET	Yes	—	Secret key for signing and verifying JWT tokens.
API_KEY	For AI Chat	—	Google Gemini API key for question generation and NLP scoring. All other features work without it.

Note: *The API_KEY is only needed for generating interview questions and NLP scoring. All other features (JD matching, eye tracking, transcription, audio analysis) work without it.*

12. Running the Application

Open two terminals side by side:

Terminal 1 — Start the Python AI Service:

```
python python_services/app.py
```

Loads Whisper model on startup (may take 30–60 seconds the first time).

Terminal 2 — Start the Node.js Server:

```
node index.js
```

Open your browser:

```
http://localhost:3000
```

The application will be available at http://localhost:3000. Register a new account, then log in to start using the platform.

13. Privacy & Security

- **Local Processing:** All video analysis (gaze detection, emotion recognition), audio transcription (Whisper), resume matching (TF-IDF), and audio quality analysis happen entirely on the user's machine. No video or audio data leaves the computer.
- **Minimal Cloud Usage:** Only text-based prompts (JD + Resume text, max 1000 characters of JD) are sent to the Google Gemini API for generating questions and scoring answers. No multimedia data is transmitted.
- **Secure Authentication:** Passwords are hashed with bcrypt (10 salt rounds). Sessions use stateless JWT tokens with 24-hour expiry. No plaintext passwords are stored.
- **Data Isolation:** All user data (profiles, sessions, transcripts, scores) is stored in the user's local PostgreSQL database. Nothing is shared across instances.
- **File Security:** Video recordings are stored locally in the uploads/ directory. The .env file containing secrets is excluded from version control via .gitignore.
- **Input Validation:** All form inputs are validated server-side (email regex, password length, required fields) before processing.

14. Future Enhancements

Potential improvements and extensions for the platform:

- **Multi-language Support:** Extend Whisper transcription and Gemini prompts to support non-English interviews.
- **Custom Question Banks:** Allow users to create, save, and share custom interview question sets.
- **Performance Trends:** Historical tracking of interview scores over time with progress charts.
- **Mock Interview Scheduling:** Allow users to schedule mock interviews with timed sessions.
- **Mobile Responsive Design:** Optimize the UI for tablet and mobile interview practice.
- **Video Recording Compression:** Implement server-side compression for smaller file sizes.
- **Peer Review Mode:** Allow users to share interview recordings for peer feedback.
- **Advanced Gaze Model:** Replace Haar Cascades with a custom-trained CNN model for more accurate gaze estimation.
- **Interview Reports:** Generate downloadable PDF reports with detailed performance summaries.
- **Cloud Deployment:** Docker containerization for one-click deployment to cloud platforms.

— End of Documentation —